

1 Цель работы

Сформировать умения разрабатывать интерфейсы, и использовать их в программах, применять механизм множественного наследования интерфейсов.

2 Постановка задачи:

Создать и реализовать интерфейсы, также использовать наследование и полиморфизм для указанных предметных областей.

В классе должны быть реализованы:

- ✓ поля (закрытые или защищённые), нужное количество;
- ✓ свойства (для каждого поля);
- ✓ конструкторы (не менее двух);
- ✓ перегруженные операции;
- ✓ методы (нужное количество);
- ✓ метод ToString();
- ✓ статические поля, если они нужны;
- ✓ статические свойства (если есть статические поля).

```
interface Корабль <- class Грузовой Корабль <- class Танкер
```

3 Код программы

IShip.cs

```
namespace LR_1
```

```
{
```

```
    interface IShip
```

```
    {
```

```
        string Name { get; set; }
```

```
        int YearBuilt { get; set; }
```

```
        double MaxSpeed { get; set; }
```

```
        static int TotalShips { get; set; } = 0;
```

```
        static int TotalShipsCount
```

```
        {
```

```
            get { return TotalShips; }
```

```
            set { TotalShips = value; }
```

```
        }
```

```
        void IncreaseShipCount();
```

```
        void Sail();
```

```
        void Dock();
```

```

        bool Equals(IShip otherShip);
        int CompareTo(IShip otherShip);
        string ToString();
    }
}

```

CargoShip.cs

```

namespace LR_1
{
    class CargoShip : IShip
    {
        public string Name { get; set; }
        public int YearBuilt { get; set; }
        public double MaxSpeed { get; set; }
        public double CargoCapacity { get; set; }
        public static int TotalShips { get; set; } = 0;
        public static int TotalShipsCount
        {
            get { return TotalShips; }
            set { TotalShips = value; }
        }

        public CargoShip()
        {
            Name = "Unknown CargoShip";
            YearBuilt = 0;
            MaxSpeed = 0;
            CargoCapacity = 0;
            IncreaseShipCount();
        }

        public CargoShip(string name, int yearBuilt, double maxSpeed, double
cargoCapacity)
        {
            Name = name;
            YearBuilt = yearBuilt;
            MaxSpeed = maxSpeed;
            CargoCapacity = cargoCapacity;
            IncreaseShipCount();
        }

        public void IncreaseShipCount() => TotalShips++;
        public void Sail() => Console.WriteLine($"{Name} is sailing at
{MaxSpeed} knots with a cargo capacity of {CargoCapacity} tons.");
        public void Dock() => Console.WriteLine($"{Name} is docking.");
    }
}

```

```

    public bool Equals(IShip otherShip)
    {
        if (otherShip == null) return false;
        return this.Name == otherShip.Name && this.YearBuilt ==
otherShip.YearBuilt;
    }
    public int CompareTo(IShip otherShip)
    {
        if (otherShip == null) return 1;
        return this.MaxSpeed.CompareTo(otherShip.MaxSpeed);
    }
    public override string ToString() => $"{Name}, Built in {YearBuilt},
Max Speed: {MaxSpeed} knots, Cargo Capacity: {CargoCapacity} tons.";
}
}

```

Tanker.cs

```

namespace LR_1
{
    class Tanker : IShip
    {
        public string Name { get; set; }
        public int YearBuilt { get; set; }
        public double MaxSpeed { get; set; }
        public double FuelCapacity { get; set; }
        public static int TotalShips { get; set; } = 0;
        public static int TotalShipsCount
        {
            get { return TotalShips; }
            set { TotalShips = value; }
        }

        public Tanker()
        {
            Name = "Unknown Tanker";
            YearBuilt = 0;
            MaxSpeed = 0;
            FuelCapacity = 0;
            IncreaseShipCount();
        }
        public Tanker(string name, int yearBuilt, double maxSpeed, double
fuelCapacity)
        {
            Name = name;

```

```

        YearBuilt = yearBuilt;
        MaxSpeed = maxSpeed;
        FuelCapacity = fuelCapacity;
        IncreaseShipCount();
    }

    public void IncreaseShipCount() => TotalShips++;
    public void Sail() => Console.WriteLine($"{Name} is sailing at
{MaxSpeed} knots with a fuel capacity of {FuelCapacity} liters.");
    public void Dock() => Console.WriteLine($"{Name} is docking.");
    public bool Equals(IShip otherShip)
    {
        if (otherShip == null) return false;
        return this.Name == otherShip.Name && this.YearBuilt ==
otherShip.YearBuilt;
    }
    public int CompareTo(IShip otherShip)
    {
        if (otherShip == null) return 1;
        return this.MaxSpeed.CompareTo(otherShip.MaxSpeed);
    }
    public override string ToString() => $"{Name}, Built in {YearBuilt},
Max Speed: {MaxSpeed} knots, Fuel Capacity: {FuelCapacity} liters.";
    }
}

```

Program.cs

```

using LR_1;

CargoShip cargoShip1 = new CargoShip("Cargo Master", 1990, 25, 50000);
Tanker tanker1 = new Tanker("Oceanic Tanker", 2005, 18, 200000);

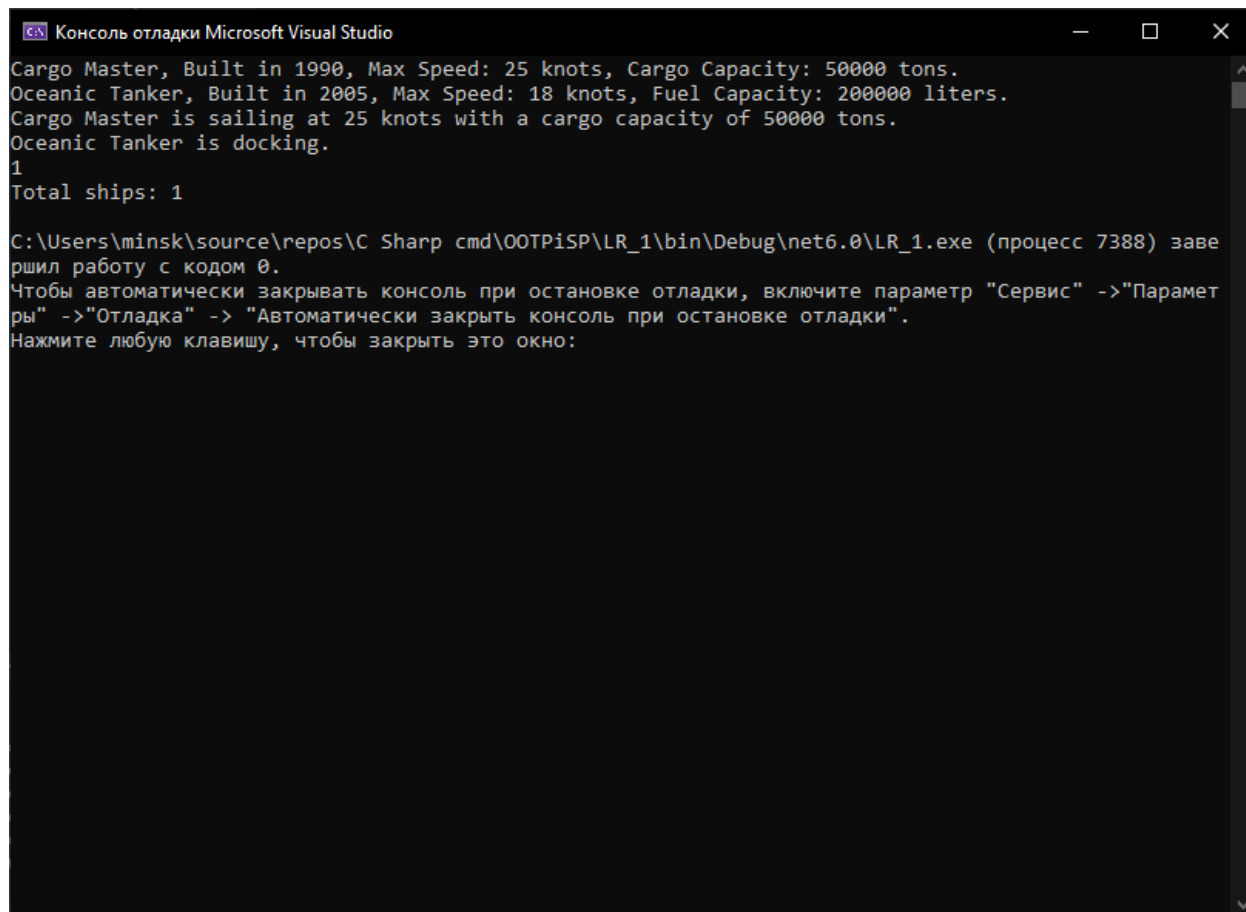
Console.WriteLine(cargoShip1.ToString());
Console.WriteLine(tanker1.ToString());

cargoShip1.Sail();
tanker1.Dock();

Console.WriteLine(cargoShip1.CompareTo(tanker1));
Console.WriteLine($"Total ships: {Tanker.TotalShipsCount}");

```

4 Результат выполнения программы (скриншоты)



```
Консоль отладки Microsoft Visual Studio
Cargo Master, Built in 1990, Max Speed: 25 knots, Cargo Capacity: 50000 tons.
Oceanic Tanker, Built in 2005, Max Speed: 18 knots, Fuel Capacity: 200000 liters.
Cargo Master is sailing at 25 knots with a cargo capacity of 50000 tons.
Oceanic Tanker is docking.
1
Total ships: 1

C:\Users\minsk\source\repos\C Sharp cmd\00TPiSP\LR_1\bin\Debug\net6.0\LR_1.exe (процесс 7388) завершил работу с кодом 0.
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

5 Выводы по работе

В ходе выполнения лабораторной работы мной были изучены умения разрабатывать интерфейсы, и использовать их в программах, применять механизм множественного наследования интерфейсов.